Pontificia Universidad Javeriana Cali

Facultad de Ingeniería
Carrera de Ingeniería de Sistemas y Computación

Parcial 1

Autor
Esteban David Muñoz Bautista
Código: 8982338

Profesor
Jefferson Amado Peña Torres

Asignatura
Sistemas Operativos

Cali, Colombia
20 de marzo de 2025

Índice

1. Introducción **2**

2. Arquitectura general del simulador **2**

3. Flujo de ejecución **2**
3.1. Aclaración importante sobre la cola inicial 3

4. Métricas de desempeño **3**

5. Análisis de casos de prueba **3**
5.1. Caso de prueba 1: `mlq001.txt` 4
5.1.1. Entrada 4
5.1.2. Salida reportada 4
5.1.3. Resumen tabular 4
5.1.4. Discusión de resultados 5
5.2. Caso de prueba 2: `mlq002.txt` 5
5.2.1. Entrada 5
5.2.2. Salida reportada 5
5.2.3. Resumen tabular 6
5.2.4. Discusión de resultados 6

6. Comparación general entre los casos **6**

7. Conclusiones **7**

1. Introducción

El presente informe documenta el funcionamiento de un simulador de planificación de procesos basado en *Multi-Level Feedback Queue* (MLFQ). Este tipo de planificador combina varias colas con distintas políticas de ejecución para buscar un equilibrio entre buen tiempo de respuesta y eficiencia global del sistema.

En este trabajo se toma como referencia la estructura del reporte base suministrado y se adapta su contenido a los archivos de entrada `mlq001.txt` y `mlq002.txt`, junto con las salidas reales proporcionadas para el **Esquema A**. Dicho esquema utiliza la siguiente configuración:

Esquema	Nivel 1	Nivel 2	Nivel 3	Nivel 4
A	RR(1)	RR(3)	RR(4)	SJF

El objetivo del informe es describir la arquitectura general del simulador, explicar el flujo de ejecución y analizar dos casos de prueba concretos a partir de sus entradas y métricas resultantes.

2. Arquitectura general del simulador

De acuerdo con el reporte base, el proyecto se encuentra organizado alrededor de los siguientes componentes:

- **Proceso**: encapsula la información de cada proceso, como tiempos de llegada, ráfaga, tiempo de finalización, espera, respuesta y retorno.
- **RR**: implementa las colas *Round Robin* para los tres primeros niveles del MLFQ, cada una con su propio quantum.
- **SJF**: implementa la cola del último nivel mediante la política *Shortest Job First* no expropiativa.
- **MLFQ**: coordina las colas, decide qué nivel tiene prioridad y mueve los procesos a niveles inferiores cuando consumen por completo su quantum.
- **Simulador**: mantiene el reloj global, inyecta los procesos al sistema y al final calcula e imprime las métricas.
- **main.cpp**: lee el archivo de entrada y da inicio a la simulación.

Esta organización modular facilita separar responsabilidades y permite razonar con claridad sobre la lógica de planificación implementada.

3. Flujo de ejecución

El flujo general del programa puede resumirse así:

1. Se lee un archivo de texto con el formato `label; BT; AT; Q; P`.
2. Se crean los procesos y se ordenan según su tiempo de llegada.
3. El simulador avanza en tiempo discreto.
4. En cada instante se verifican nuevas llegadas y se entrega el control al MLFQ.
5. El MLFQ selecciona la cola no vacía de mayor prioridad.
6. Si un proceso agota el quantum de su nivel sin terminar, desciende a la siguiente cola.
7. Cuando un proceso termina, se registran sus métricas finales.

3.1. Aclaración importante sobre la cola inicial

El reporte base resalta un detalle relevante de la implementación: aunque el archivo de entrada incluye un campo `Q`, la lógica descrita del simulador inyecta las llegadas en la primera cola `RR(1)` y luego las degrada progresivamente según el consumo de quantum. Por ello, en este informe el valor de `Q` se conserva como parte del archivo de entrada, pero la discusión de resultados se interpreta a la luz del comportamiento real documentado para el código.

4. Métricas de desempeño

Las métricas observadas en la salida del simulador son las siguientes:

- **CT (Completion Time):** instante en que el proceso termina.
- **TAT (Turn Around Time):** tiempo total transcurrido desde la llegada hasta la finalización.
- **WT (Waiting Time):** tiempo acumulado de espera del proceso.
- **RT (Response Time):** tiempo transcurrido entre la llegada y la primera vez que recibe CPU.

Además, el simulador reporta los promedios del sistema para estas métricas, lo cual permite comparar el comportamiento general de los casos analizados.

5. Análisis de casos de prueba

En esta sección se presentan los dos archivos de entrada suministrados y las salidas reales obtenidas para el Esquema A.

5.1. Caso de prueba 1: `mlq001.txt`

5.1.1. Entrada

Listing 1: Archivo de entrada mlq001.txt

```
# Archivo: mlq001.txt
# etiqueta; burst time (BT); arrival time (AT); Queue (Q); Priority (5 > 1)
A; 6; 0; 1; 5
B; 9; 0; 1; 4
C; 10; 0; 2; 3
D; 15; 0; 2; 3
E; 8; 0; 3; 2
```

5.1.2. Salida reportada

Listing 2: Salida del simulador para mlq001.txt

```
P       AT      BT      Q       CT      TAT     WT      RT
-----------------------------------------------------------
A       0       6       1       22      22      16      0
B       0       9       1       39      39      30      1
C       0       10      2       41      41      31      2
D       0       15      2       48      48      33      3
E       0       8       3       38      38      30      4
-----PROMEDIOS-----
Completion Time: 37.6
Turn Around Time: 37.6
Waiting Time: 28
Response Time: 2
```

5.1.3. Resumen tabular

P	AT	BT	Q	CT	TAT	WT	RT
A	0	6	1	22	22	16	0
B	0	9	1	39	39	30	1
C	0	10	2	41	41	31	2
D	0	15	2	48	48	33	3
E	0	8	3	38	38	30	4

Promedios del caso 1. CT promedio = 37.6, TAT promedio = 37.6, WT promedio = 28.0 y RT promedio = 2.0.

5.1.4. Discusión de resultados

En este caso todos los procesos llegan en el tiempo 0, por lo que la competencia por CPU inicia inmediatamente. Los tiempos de respuesta crecen de forma escalonada entre 0 y 4, lo que sugiere una atención inicial corta y repartida entre los procesos más prioritarios.

También puede observarse que los procesos con mayor ráfaga tienden a terminar más tarde y a acumular mayor tiempo de espera. Esto se refleja especialmente en `D`, que tiene el mayor *burst time* y presenta los valores más altos de `CT` y `WT`. En contraste, `A`, al tener una ráfaga más pequeña, finaliza antes y obtiene las mejores métricas globales del grupo.

La diferencia entre los procesos `B`, `C` y `E` muestra que el comportamiento del algoritmo no depende únicamente del *burst time*, sino también del orden en que cada proceso recibe CPU en los niveles superiores y de cómo va descendiendo a colas de menor prioridad.

5.2. Caso de prueba 2: `mlq002.txt`

5.2.1. Entrada

Listing 3: Archivo de entrada mlq002.txt

```
# Archivo: mlq002.txt
# 09102024
p1; 20; 0; 1; 5
p2; 10; 2; 1; 4
p3; 15; 4; 2; 3
p4; 5; 6; 3; 2
p5; 8; 8; 3; 1
```

5.2.2. Salida reportada

Listing 4: Salida del simulador para mlq002.txt

```
P       AT      BT      Q       CT      TAT     WT      RT
-------------------------------------------------------------
p1      0       20      1       58      58      38      0
p2      2       10      1       39      37      27      4
p3      4       15      2       46      42      27      5
p4      6       5       3       33      27      22      6
p5      8       8       3       37      29      21      10
-----PROMEDIOS-----
Completion Time: 42.6
Turn Around Time: 38.6
Waiting Time: 27
Response Time: 5
```

5.2.3. Resumen tabular

P	AT	BT	Q	CT	TAT	WT	RT
p1	0	20	1	58	58	38	0
p2	2	10	1	39	37	27	4
p3	4	15	2	46	42	27	5
p4	6	5	3	33	27	22	6
p5	8	8	3	37	29	21	10

Promedios del caso 2. CT promedio = 42.6, TAT promedio = 38.6, WT promedio = 27.0 y RT promedio = 5.0.

5.2.4. Discusión de resultados

A diferencia del primer caso, aquí las llegadas son escalonadas. Esto genera una dinámica distinta: el proceso `p1` recibe CPU de inmediato y por eso tiene `RT = 0`, mientras que los procesos que llegan después deben esperar a que el sistema vuelva a atender la cola superior. El efecto más claro se observa en `p5`, cuyo tiempo de respuesta asciende a 10.

Aunque `p4` tiene la ráfaga más corta del conjunto, no obtiene el mejor tiempo de respuesta porque llega tarde al sistema. Sin embargo, sí logra una finalización relativamente temprana con respecto a procesos más largos como `p1` y `p3`. Por otro lado, `p1` concentra los mayores valores de `CT`, `TAT` y `WT` debido a que su ráfaga es la más larga y termina atravesando más etapas del esquema de colas.

Este caso deja ver con mayor claridad el efecto combinado entre la política MLFQ y los tiempos de llegada: no basta con conocer la duración del proceso; también influye el momento en que entra al sistema y cuánto tiempo permanece compitiendo en cada nivel.

6. Comparación general entre los casos

A partir de los promedios reportados se puede observar lo siguiente:

Caso	CT prom.	TAT prom.	WT prom.	RT prom.
`mlq001.txt`	37.6	37.6	28.0	2.0
`mlq002.txt`	42.6	38.6	27.0	5.0

El primer caso presenta un mejor tiempo de respuesta promedio porque todos los procesos llegan desde el inicio y el sistema reparte rápidamente la atención inicial. En el segundo caso, las llegadas escalonadas incrementan el tiempo de respuesta promedio, especialmente para los procesos tardíos. No obstante, el tiempo de espera promedio del caso 2 es ligeramente menor,

lo que sugiere que las llegadas distribuidas en el tiempo pueden aliviar parte de la congestión inicial del sistema.

7. Conclusiones

El análisis realizado permite concluir que el simulador MLFQ implementado con el Esquema A ofrece una forma clara de estudiar la interacción entre *Round Robin* y *Shortest Job First* dentro de una jerarquía de colas.

En el caso `mlq001.txt` se aprecia con facilidad el efecto de la competencia simultánea por CPU: los procesos largos acumulan más espera y terminan en instantes mayores. En el caso `mlq002.txt`, las llegadas escalonadas muestran que el tiempo de respuesta depende fuertemente del instante de arribo, incluso cuando algunos procesos tienen ráfagas relativamente cortas.

En conjunto, ambos ejemplos evidencian que MLFQ no solo clasifica procesos por duración, sino también por comportamiento observado durante la ejecución. Esa es precisamente una de sus fortalezas principales: adaptar la prioridad de cada proceso a medida que el sistema aprende cómo consume CPU.